E-Bike Monitor — Source Code

Formatted PDF copy of the uploaded Arduino/C++ source file: ebike_monitor-1.ino

```
/* ============================================================
 * AI-Powered Multi-Brand E-Bike Recommendation and Smart
 * Monitoring System  -  Monitoring Module (ESP32)
 *
 * Review-2 complete prototype.
 *
 * HARDWARE FITTED
 *   ESP32 DevKit V1
 *   2x DS18B20      GPIO4  (one shared bus, 4.7k pull-up to 3V3)
 *   MPU6050         I2C    (mounted VERTICALLY - orientation is learned)
 *   BMP280          I2C    (0x76, some boards 0x77)
 *   0.96" OLED      I2C    (0x3C)
 *   Active buzzer   GPIO5
 *   Red LED         GPIO25 (through 220R)
 *   Green LED       GPIO26 (through 220R)
 *   INA219          I2C    (0x40) - battery voltage, current, power
 *   NEO-6M GPS      UART2  (GPS TX -> GPIO16, GPS RX -> GPIO17, 9600 baud)
 *
 * Every sensor is detected at boot. One that is missing has its features
 * switched off (never faked) and is retried every 10 s.
 *
 * BACKEND
 *   Supabase  - telemetry every 3 s, alerts on state change
 *   Telegram  - instant message on every warning and critical alert
 *
 * LIBRARIES
 *   OneWire, DallasTemperature, Adafruit BMP280, Adafruit SSD1306,
 *   Adafruit Unified Sensor, Adafruit INA219, TinyGPSPlus, ArduinoJson
 *
 * SET BEFORE UPLOAD
 *   WIFI_SSID, WIFI_PASS, TELEGRAM_TOKEN, TELEGRAM_CHAT_ID
 * ============================================================ */

#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_INA219.h>
#include <TinyGPSPlus.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// ============================================================
//  CONFIGURATION
// ============================================================

#define USE_SUPABASE true
#define USE_TELEGRAM true

// ---- Wi-Fi ----
const char* WIFI_SSID = "iqoo owner";
const char* WIFI_PASS = "54321678";

// ---- Supabase ----
const char* SUPABASE_URL = "https://juhygjhhoittuegzotud.supabase.co";
const char* SUPABASE_KEY = "sb_publishable_1Z3KaX6rCf04e8Y2jAKGiw_ts4LFyBA";
const char* DEVICE_ID    = "ebike-01";

// ---- Telegram ----
// Get the token from @BotFather. Get the chat id by messaging your bot
// once, then opening  https://api.telegram.org/bot<TOKEN>/getUpdates
const char* TELEGRAM_TOKEN   = "8738607596:AAHZGn3xyHY8ePpM5m0WDYoJunxjDizsnoc";
const char* TELEGRAM_CHAT_ID = "8328336918";

// ---- Pins ----
#define PIN_ONEWIRE     4
#define PIN_BUZZER      5
#define PIN_LED_RED    25
```

```
#define PIN_LED_GREEN  26
// I2C on the default GPIO21 (SDA) / GPIO22 (SCL)

// ---- Timing ----
const unsigned long READ_INTERVAL     = 500;    // sensor + display refresh
const unsigned long SUPABASE_INTERVAL = 3000;   // one telemetry row
const unsigned long TELEGRAM_COOLDOWN = 60000;  // min gap for the same code

// ---- Thresholds ----
// Each alert has a trigger value and a lower clear value, so a reading
// sitting on the limit cannot make the alert flicker on and off.
float TEMP_MOTOR_WARN  = 55.0, TEMP_MOTOR_WARN_CLR = 52.0;
float TEMP_MOTOR_CRIT  = 70.0, TEMP_MOTOR_CRIT_CLR = 66.0;
float TEMP_BATT_WARN   = 45.0, TEMP_BATT_WARN_CLR  = 42.0;
float TEMP_BATT_CRIT   = 55.0, TEMP_BATT_CRIT_CLR  = 51.0;
float VIBRATION_WARN   = 2.5,  VIBRATION_WARN_CLR  = 1.8;
float VIBRATION_CRIT   = 6.0,  VIBRATION_CRIT_CLR  = 4.5;
float IMPACT_THRESHOLD = 12.0;
float TILT_THRESHOLD   = 30.0;   // degrees away from the mounted position
float IDLE_VIBRATION   = 0.8;
unsigned long ARM_DELAY = 30000; // stillness before parking mode arms
// ---- INA219 power limits ----
// Voltage limits are a percentage of the supply seen at boot, so the same
// firmware works on a 5 V USB supply, a 9 V battery or a 12 V pack. Fixed
// 11 V / 10 V limits would put a 5 V demo supply into permanent CRITICAL.
// Set VOLT_NOMINAL (e.g. 12.6) to use a fixed pack voltage instead.
float VOLT_NOMINAL      = 0;      // 0 = learn from the supply
float VOLT_WARN_PCT     = 0.85, VOLT_WARN_CLR_PCT = 0.88;
float VOLT_CRIT_PCT     = 0.75, VOLT_CRIT_CLR_PCT = 0.78;
float VOLT_NO_SUPPLY    = 0.5;    // below this nothing is connected - not an alarm
float CURRENT_MAX_WARN  = 2000.0; // mA
float CURRENT_MAX_CRIT  = 3000.0; // mA (the module's shunt tops out near 3.2 A)

// ---- NEO-6M GPS ----
#define PIN_GPS_RX 16             // ESP32 receives here  <- GPS TX
#define PIN_GPS_TX 17             // ESP32 transmits here -> GPS RX
const uint32_t GPS_BAUD = 9600;
float OVERSPEED_KMH     = 25.0;   // legal limit for low-speed e-bikes in India
float OVERSPEED_CLR_KMH = 22.0;
float GEOFENCE_M        = 50.0;   // moved this far while parked = theft
float GPS_MAX_HDOP      = 5.0;    // speed and geofence ignored on a poorer fix
const unsigned long GPS_FIX_MAX_AGE = 5000;

const uint8_t CONFIRM_SAMPLES = 2;         // reads a condition must hold
const unsigned long IMPACT_LATCH_MS = 8000; // impact stays visible this long

// ============================================================
//  OBJECTS AND STATE
// ============================================================

OneWire oneWire(PIN_ONEWIRE);
DallasTemperature sensors(&oneWire);
Adafruit_BMP280 bmp;
Adafruit_SSD1306 display(128, 64, &Wire, -1);
DeviceAddress addrMotor, addrBattery;

// ---- INA219 state ----
Adafruit_INA219* ina = nullptr;     // created at the address actually found
uint8_t inaAddr = 0;
bool    inaOk = false;
uint8_t inaFailStreak = 0;
float   voltNominal = 0;            // learned supply voltage
unsigned long supplySince = 0, noSupplySince = 0;

// ---- GPS state ----
TinyGPSPlus gps;
HardwareSerial GPSSerial(2);
bool    gpsOk = false, gpsEverSeen = false;
unsigned long gpsLastByte = 0;
double  lastFixLat = NAN, lastFixLng = NAN;
unsigned long lastFixAt = 0;
double  parkLat = NAN, parkLng = NAN;
double  tripLat = NAN, tripLng = NAN;
float   tripKm = 0;

// Supabase table may predate the GPS/power columns. If it rejects them,
// fall back to the original fields rather than losing every upload.
bool    extendedSchema = true;
String  lastHttpBody = "";
```

```
// ---- I2C sensor presence ------------------------------------------
// A sensor that did not answer at boot used to be read anyway, and the
// garbage it returned (209 m/s^2) set off permanent impact alarms. Now
// each sensor is only used while it is actually answering. A missing
// sensor switches its features off, and is re-probed every 10 s so it
// recovers by itself if a loose wire is pushed back in.
uint8_t mpuAddr = 0, mpuWhoAmI = 0;
bool    mpuOk = false;
uint8_t mpuFailStreak = 0;
uint8_t bmpAddr = 0, bmpChipId = 0;
bool    bmpOk = false;
uint8_t bmpFailStreak = 0;
unsigned long lastProbe = 0;
const unsigned long REPROBE_MS = 10000;
String  pendingHw = "";                 // hardware events waiting to be sent

// Several sensors can drop out at once (one loose ground wire takes both),
// so events are appended rather than overwriting each other.
void queueHw(const String& msg) {
  if (pendingHw.length()) pendingHw += "\n\n";
  pendingHw += msg;
}

bool i2cPresent(uint8_t addr) {
  Wire.beginTransmission(addr);
  return Wire.endTransmission() == 0;
}

bool i2cReadReg(uint8_t addr, uint8_t reg, uint8_t* buf, uint8_t len) {
  Wire.beginTransmission(addr);
  Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(addr, len) != len) return false;
  for (uint8_t i = 0; i < len; i++) buf[i] = Wire.read();
  return true;
}

bool i2cWriteReg(uint8_t addr, uint8_t reg, uint8_t val) {
  Wire.beginTransmission(addr);
  Wire.write(reg);
  Wire.write(val);
  return Wire.endTransmission() == 0;
}

String hex2(uint8_t v) {
  const char* d = "0123456789ABCDEF";
  String out = "0x";
  out += d[v >> 4];
  out += d[v & 0x0F];
  return out;
}

// Name whatever answers on the bus, using each chip's ID register.
String identifyI2C(uint8_t addr) {
  uint8_t id = 0;
  if (addr == 0x68 || addr == 0x69) {
    if (!i2cReadReg(addr, 0x75, &id, 1)) return "IMU (no WHO_AM_I reply)";
    if (id == 0x68) return "MPU6050 (WHO_AM_I " + hex2(id) + ")";
    if (id == 0x70) return "MPU6500 clone (WHO_AM_I " + hex2(id) + ")";
    if (id == 0x71) return "MPU9250 (WHO_AM_I " + hex2(id) + ")";
    if (id == 0x73) return "MPU9255 (WHO_AM_I " + hex2(id) + ")";
    return "MPU-compatible? (WHO_AM_I " + hex2(id) + ")";
  }
  if (addr == 0x76 || addr == 0x77) {
    if (!i2cReadReg(addr, 0xD0, &id, 1)) return "pressure sensor (no ID reply)";
    if (id == 0x58) return "BMP280 (chip ID " + hex2(id) + ")";
    if (id == 0x56 || id == 0x57) return "BMP280 sample (chip ID " + hex2(id) + ")";
    if (id == 0x60) return "BME280 (chip ID " + hex2(id) + ")";
    if (id == 0x55) return "BMP180 - NOT supported (chip ID " + hex2(id) + ")";
    return "unknown (chip ID " + hex2(id) + ")";
  }
  if (addr == 0x3C || addr == 0x3D) return "SSD1306 OLED";
  if (addr >= 0x40 && addr <= 0x4F) return "INA219?";
  return "unknown device";
}

uint8_t scanI2C() {
  uint8_t found = 0;
  Serial.println("---------------------------------------");
```

```
  Serial.println("  I2C BUS SCAN (SDA 21, SCL 22)");
  Serial.println("---------------------------------------");
  for (uint8_t a = 1; a < 127; a++) {
    if (!i2cPresent(a)) continue;
    found++;
    Serial.print("  ");
    Serial.print(hex2(a));
    Serial.print("  ");
    Serial.println(identifyI2C(a));
  }
  if (!found) Serial.println("  nothing found - check SDA/SCL and 3V3/GND");
  Serial.println("  expected: 0x3C OLED, 0x68 MPU6050, 0x76 BMP280");
  Serial.println("---------------------------------------");
  return found;
}

// Raw accelerometer read. Works for the MPU6050 and the clones sold on
// GY-521 boards (MPU6500/9250), whose accelerometer registers match.
bool mpuRead(float &x, float &y, float &z) {
  if (!mpuAddr) return false;
  uint8_t b[6];
  if (!i2cReadReg(mpuAddr, 0x3B, b, 6)) return false;
  int16_t rx = (int16_t)((b[0] << 8) | b[1]);
  int16_t ry = (int16_t)((b[2] << 8) | b[3]);
  int16_t rz = (int16_t)((b[4] << 8) | b[5]);
  // All 0xFF or all zero means the bus returned nothing real.
  if ((rx == -1 && ry == -1 && rz == -1) || (rx == 0 && ry == 0 && rz == 0)) return false;
  const float k = 9.80665f / 4096.0f;           // +/-8 g range: 4096 LSB per g
  x = rx * k; y = ry * k; z = rz * k;
  // A +/-8 g sensor cannot report more than ~136 m/s^2 in total.
  if (sqrt(x * x + y * y + z * z) > 140.0f) return false;
  return true;
}

bool mpuInit() {
  const uint8_t addrs[2] = {0x68, 0x69};        // 0x69 when AD0 is pulled high
  for (uint8_t i = 0; i < 2; i++) {
    uint8_t a = addrs[i], id = 0;
    if (!i2cPresent(a)) continue;
    if (!i2cReadReg(a, 0x75, &id, 1)) continue;
    i2cWriteReg(a, 0x6B, 0x00);                 // wake from sleep
    delay(50);
    i2cWriteReg(a, 0x1C, 0x10);                 // accelerometer +/-8 g
    if (id == 0x68) i2cWriteReg(a, 0x1A, 0x04); // MPU6050 low-pass ~21 Hz
    else            i2cWriteReg(a, 0x1D, 0x04); // MPU6500/9250 accel low-pass
    delay(20);
    mpuAddr = a;
    mpuWhoAmI = id;
    float x, y, z;
    if (!mpuRead(x, y, z)) { mpuAddr = 0; continue; }
    float m = sqrt(x * x + y * y + z * z);
    if (m < 5.0f || m > 15.0f) {                // at rest it must read ~1 g
      Serial.print("  IMU at "); Serial.print(hex2(a));
      Serial.print(" reads "); Serial.print(m, 1);
      Serial.println(" m/s2 at rest, expected ~9.8 - rejected");
      mpuAddr = 0;
      continue;
    }
    mpuFailStreak = 0;
    return true;
  }
  mpuAddr = 0;
  return false;
}

bool bmpInit() {
  const uint8_t addrs[2] = {0x76, 0x77};
  for (uint8_t i = 0; i < 2; i++) {
    uint8_t a = addrs[i], id = 0;
    if (!i2cPresent(a)) continue;
    if (!i2cReadReg(a, 0xD0, &id, 1)) continue;
    // BMP280 and BME280 share the temperature/pressure registers.
    bool known = (id == 0x58 || id == 0x56 || id == 0x57 || id == 0x60);
    if (!known) continue;
    if (bmp.begin(a, id)) {
      bmpAddr = a;
      bmpChipId = id;
      bmpFailStreak = 0;
      return true;
```

```
    }
  }
  bmpAddr = 0;
  return false;
}

// INA219 can sit at 0x40-0x45 depending on its A0/A1 solder jumpers.
bool inaInit() {
  for (uint8_t a = 0x40; a <= 0x45; a++) {
    if (!i2cPresent(a)) continue;
    if (ina) { delete ina; ina = nullptr; }
    ina = new Adafruit_INA219(a);
    if (!ina->begin()) { delete ina; ina = nullptr; continue; }
    inaAddr = a;
    inaFailStreak = 0;
    return true;
  }
  inaAddr = 0;
  return false;
}

// Remember the supply voltage once it has been steady for 2 s. Unplugging
// the supply forgets it, so a different source is learned fresh.
void learnSupply(float v) {
  if (VOLT_NOMINAL > 0) { voltNominal = VOLT_NOMINAL; return; }
  if (v < VOLT_NO_SUPPLY) {
    supplySince = 0;
    if (!noSupplySince) noSupplySince = millis();
    if (voltNominal > 0 && millis() - noSupplySince > 3000) {
      voltNominal = 0;
      Serial.println("[POWER] supply removed - will learn the next one");
    }
    return;
  }
  noSupplySince = 0;
  if (!supplySince) supplySince = millis();
  if (voltNominal == 0 && millis() - supplySince > 2000) {
    voltNominal = v;
    Serial.print("[POWER] supply learned: "); Serial.print(v, 2); Serial.println(" V
nominal");
  } else if (voltNominal > 0 && v > voltNominal * 1.2) {
    voltNominal = v;                // a higher-voltage source was connected
    Serial.print("[POWER] higher supply, relearned: "); Serial.print(v, 2); Serial.println("
 V");
  }
}

// Drain the GPS UART. Called every loop pass so sentences are not lost.
void serviceGPS() {
  while (GPSSerial.available()) {
    gps.encode(GPSSerial.read());
    gpsLastByte = millis();
  }
  bool receiving = gpsLastByte && millis() - gpsLastByte < 3000;
  if (receiving != gpsOk) {
    gpsOk = receiving;
    if (!receiving) {
      Serial.println("[SENSOR] GPS stopped sending data");
      queueHw("NEO-6M GPS stopped sending data. Location, speed and geofence are off. Check
GPS TX -> GPIO16 and its power.");
    } else if (gpsEverSeen) {
      Serial.println("[SENSOR] GPS data is back");
      queueHw("NEO-6M GPS reconnected.");
    }
    if (receiving) gpsEverSeen = true;
  }
}

String mapsLink(double lat, double lng) {
  return "https://maps.google.com/?q=" + String(lat, 6) + "," + String(lng, 6);
}

void reportSensor(const char* name, bool ok, uint8_t addr, uint8_t id, const char* lost) {
  Serial.print(name);
  if (ok) {
    Serial.print("ok at "); Serial.print(hex2(addr));
    Serial.print(" (ID "); Serial.print(hex2(id)); Serial.println(")");
  } else {
    Serial.print("NOT DETECTED - "); Serial.println(lost);
```

```
  }
}


enum Status { NORMAL = 0, WARNING = 1, CRITICAL = 2 };

struct Readings {
  float motorTemp, batteryTemp, ambientTemp, pressure;
  float vibration, tiltAngle;
  bool  impactDetected;
  float voltage, current, power;
  bool  noSupply;
  bool  gpsFix;
  double lat, lng;
  float speed, altitude, hdop, geoDistance;
  int   sats;
  Status status;
  String reason, code;
  unsigned long timestamp;
} r;

// Gravity vector captured at boot. Tilt is measured against THIS, so the
// board can be mounted vertically, flat, or at any fixed angle.
float refX = 0, refY = 0, refZ = 9.81, restingAccel = 9.81;

bool parkingArmed = false;
unsigned long stillSince = 0, lastRead = 0, lastUpload = 0;
uint16_t uploadOk = 0, uploadFail = 0, tgOk = 0, tgFail = 0;

uint8_t cMotorW = 0, cMotorC = 0, cBattW = 0, cBattC = 0;
uint8_t cVibW = 0, cVibC = 0, cParkMove = 0;
uint8_t cVoltW = 0, cVoltC = 0, cCurW = 0, cCurC = 0, cOver = 0, cGeo = 0;
bool    fVoltW = false, fVoltC = false, fCurW = false, fCurC = false, fOver = false;
bool fMotorW = false, fMotorC = false, fBattW = false, fBattC = false;
bool fVibW = false, fVibC = false;
unsigned long impactUntil = 0, alertScreenUntil = 0;
uint16_t warnCount = 0, critCount = 0;
Status prevStatus = NORMAL;

String   lastTgCode = "";
unsigned long lastTgTime = 0;

// A network send blocks for up to 8 s. Doing that inside evaluateStatus()
// would freeze the buzzer mid-beep and stall the display exactly when the
// alert fires, so the send is queued and dispatched from loop() once the
// audible pattern has finished playing.
struct PendingAlert {
  bool   active = false;
  Status level  = NORMAL;
  String code, reason;
} pending;

bool buzzerIdle();

// ---- Forward declarations ----
const char* statusName(Status s);
String buildJSON();
void publishSerial();
void bootAnimation();
void drawWheel(int cx, int cy, int radius, float angle);
void drawSmallWheel(int cx, int cy, int radius, float angle, int color);
void calibrationScreen();
void calibrateOrientation();
void sensorCheckScreen();
void readySplash();
void connectWiFi();
void reportBackends();
void readSensors();
void updateParkingState();
void evaluateStatus();
void updateOutputs();
void setLEDs(Status s);
void drawLiveScreen();
void drawAlertScreen();
int  supabasePost(const char* table, const String& body);
void uploadTelemetry();
void uploadAlert(Status level, const String& code, const String& reason);
bool telegramSend(const String& text);
void telegramAlert(Status level, const String& code, const String& reason);
```

```
// ============================================================
//  BUZZER - non-blocking pattern player
// ============================================================
struct BuzzerPattern { uint16_t freq, onMs, offMs; uint8_t reps; };
BuzzerPattern buzzPat = {0, 0, 0, 0};
uint8_t buzzStep = 0;
bool buzzOn = false;
unsigned long buzzNext = 0;

void startBuzzer(uint16_t freq, uint16_t onMs, uint16_t offMs, uint8_t reps) {
  buzzPat = {freq, onMs, offMs, reps};
  buzzStep = 0; buzzOn = false; buzzNext = 0;
}

void stopBuzzer() { noTone(PIN_BUZZER); buzzStep = 255; }

bool buzzerIdle() { return buzzStep >= buzzPat.reps * 2; }

void serviceBuzzer() {
  if (buzzStep >= buzzPat.reps * 2) { noTone(PIN_BUZZER); return; }
  if (millis() < buzzNext) return;
  if (!buzzOn) {
    tone(PIN_BUZZER, buzzPat.freq);
    buzzOn = true;
    buzzNext = millis() + buzzPat.onMs;
  } else {
    noTone(PIN_BUZZER);
    buzzOn = false;
    buzzNext = millis() + buzzPat.offMs;
  }
  buzzStep++;
}

// ============================================================
//  SETUP
// ============================================================
void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.println();
  Serial.println("======================================");
  Serial.println("  E-BIKE SMART MONITORING SYSTEM");
  Serial.println("  Review-2 Prototype");
  Serial.println("======================================");
  Serial.println("Initialising sensors...");

  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_LED_RED, OUTPUT);
  pinMode(PIN_LED_GREEN, OUTPUT);

  Wire.begin(21, 22);

  sensors.begin();
  Serial.print("DS18B20  : ");
  Serial.print(sensors.getDeviceCount());
  Serial.println(" device(s) found");
  if (!sensors.getAddress(addrMotor, 0))   Serial.println("  WARNING: sensor #0 missing");
  if (!sensors.getAddress(addrBattery, 1)) Serial.println("  WARNING: sensor #1 missing");
  sensors.setResolution(10);

  scanI2C();
  mpuOk = mpuInit();
  reportSensor("MPU6050  : ", mpuOk, mpuAddr, mpuWhoAmI,
               "vibration, tilt, impact and parking alerts are OFF");
  bmpOk = bmpInit();
  reportSensor("BMP280   : ", bmpOk, bmpAddr, bmpChipId,
               "ambient temperature and pressure are OFF");

  Serial.print("OLED     : ");
  Serial.println(display.begin(SSD1306_SWITCHCAPVCC, 0x3C) ? "ok" : "NOT FOUND");
  display.setTextColor(SSD1306_WHITE);

  inaOk = inaInit();
  Serial.print("INA219   : ");
  if (inaOk) { Serial.print("ok at "); Serial.println(hex2(inaAddr)); }
  else Serial.println("NOT DETECTED - voltage, current and power are OFF");

  // NEO-6M on UART2. A larger buffer rides out the pause while an HTTPS
  // upload blocks the loop; at 9600 baud 2 KB is about 2 seconds of data.
```

```
  GPSSerial.setRxBufferSize(2048);
  GPSSerial.begin(GPS_BAUD, SERIAL_8N1, PIN_GPS_RX, PIN_GPS_TX);
  unsigned long gpsWait = millis();
  while (millis() - gpsWait < 2500 && !gpsOk) { serviceGPS(); delay(20); }
  Serial.print("NEO-6M   : ");
  if (!gpsOk) {
    Serial.println("NOT DETECTED - no data on GPIO16 (check GPS TX wire and power)");
  } else if (gps.passedChecksum() == 0) {
    Serial.println("receiving data but no valid sentences - check GPS_BAUD");
  } else {
    Serial.println("ok, receiving NMEA - fix needs open sky, can take 1-5 min");
  }

  bootAnimation();
  if (!mpuOk || !bmpOk || !inaOk || !gpsOk) sensorCheckScreen();

  calibrationScreen();
  calibrateOrientation();
  stillSince = millis();

  connectWiFi();
  reportBackends();

#if USE_TELEGRAM
  if (WiFi.status() == WL_CONNECTED) {
    telegramSend(String("\xF0\x9F\x9A\xB4 *E-Bike Monitor online*\n") +
                 "Device: `" + DEVICE_ID + "`\n" +
                 "IP: `" + WiFi.localIP().toString() + "`\n" +
                 "MPU6050: " + (mpuOk ? "ok" : "*NOT DETECTED* - motion alerts off") + "\n"
+
                 "BMP280: " + (bmpOk ? "ok" : "*NOT DETECTED* - ambient off") + "\n" +
                 "INA219: " + (inaOk ? "ok" : "*NOT DETECTED* - power off") + "\n" +
                 "GPS: " + (gpsOk ? "receiving, waiting for fix" : "*NOT DETECTED* -
location off"));
  }
#endif

  readySplash();
}

// Learn the mounted orientation. Whatever direction gravity points now
// becomes "upright" - so a vertical MPU6050 reads 0 degrees of tilt.
void calibrateOrientation() {
  Serial.println("--------------------------------------");
  Serial.println("  ORIENTATION CALIBRATION");
  Serial.println("--------------------------------------");
  if (!mpuOk) {
    Serial.println("Skipped - MPU6050 not detected.");
    Serial.println("--------------------------------------");
    return;
  }
  float sx = 0, sy = 0, sz = 0, sMag = 0;
  int good = 0;
  for (int i = 0; i < 100; i++) {
    float x, y, z;
    if (mpuRead(x, y, z)) {
      sx += x; sy += y; sz += z;
      sMag += sqrt(x * x + y * y + z * z);
      good++;
    }
    delay(8);
  }
  if (good < 50) {
    Serial.print("Only "); Serial.print(good);
    Serial.println("/100 reads succeeded - MPU6050 unreliable, disabled.");
    mpuOk = false;
    Serial.println("--------------------------------------");
    return;
  }
  refX = sx / good; refY = sy / good; refZ = sz / good;
  restingAccel = sMag / good;

  Serial.print("Gravity  : X="); Serial.print(refX, 2);
  Serial.print("  Y=");          Serial.print(refY, 2);
  Serial.print("  Z=");          Serial.println(refZ, 2);
  Serial.print("Mounting : ");
  if (fabs(refZ) > fabs(refX) && fabs(refZ) > fabs(refY))
    Serial.println("FLAT (gravity on Z)");
  else if (fabs(refY) > fabs(refX))
```

```
    Serial.println("VERTICAL (gravity on Y)");
  else
    Serial.println("VERTICAL (gravity on X)");
  Serial.println("Tilt is measured from this position.");
  Serial.println("--------------------------------------");
}

// Shown at boot when a sensor is missing, so it is obvious on the device
// itself rather than only in the serial monitor.
void sensorCheckScreen() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(4, 4);
  display.println("SENSOR CHECK");
  display.drawLine(4, 14, 124, 14, SSD1306_WHITE);
  display.setCursor(4, 18);
  display.print("MPU6050  "); display.println(mpuOk ? "OK" : "MISSING");
  display.setCursor(4, 27);
  display.print("BMP280   "); display.println(bmpOk ? "OK" : "MISSING");
  display.setCursor(4, 36);
  display.print("INA219   "); display.println(inaOk ? "OK" : "MISSING");
  display.setCursor(4, 45);
  display.print("NEO-6M   "); display.println(gpsOk ? "OK" : "MISSING");
  display.setCursor(4, 56);
  display.println("Rest of system runs");
  display.display();
  delay(3500);
}

void connectWiFi() {
  Serial.println();
  Serial.println("--------------------------------------");
  Serial.println("  WIFI CONNECTION");
  Serial.println("--------------------------------------");
  Serial.print("SSID     : "); Serial.println(WIFI_SSID);
  Serial.print("Status   : connecting");

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  unsigned long start = millis();
  int dots = 0;
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
    Serial.print(".");
    if (++dots % 20 == 0) Serial.print("\n           ");
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(10, 14);
    display.println("CONNECTING WIFI");
    display.setCursor(10, 28);
    display.println(WIFI_SSID);
    for (int d = 0; d < 5; d++) {
      if (d == (dots % 5)) display.fillCircle(38 + d * 12, 48, 4, SSD1306_WHITE);
      else                 display.drawCircle(38 + d * 12, 48, 4, SSD1306_WHITE);
    }
    display.display();
    delay(400);
  }
  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("Status   : CONNECTED");
    Serial.print("IP       : "); Serial.println(WiFi.localIP());
    Serial.print("Gateway  : "); Serial.println(WiFi.gatewayIP());
    Serial.print("RSSI     : "); Serial.print(WiFi.RSSI()); Serial.println(" dBm");
    Serial.print("Time     : "); Serial.print((millis() - start) / 1000.0, 1);
    Serial.println(" s");
  } else {
    Serial.println("Status   : NOT CONNECTED");
    Serial.println("Reason   : timed out after 15 s");
    Serial.println("Check    : SSID spelling, password, 2.4 GHz band");
    Serial.println("Running OFFLINE - display and buzzer still work.");
  }
  Serial.println("--------------------------------------");
}

void reportBackends() {
  Serial.println();
  Serial.println("--------------------------------------");
```

```
  Serial.println("  BACKEND CONNECTIONS");
  Serial.println("-------------------------------------");
#if USE_SUPABASE
  Serial.print("Supabase : "); Serial.println(SUPABASE_URL);
  Serial.print("Device   : "); Serial.println(DEVICE_ID);
  Serial.print("Upload   : every ");
  Serial.print(SUPABASE_INTERVAL / 1000.0, 1);
  Serial.println(" s -> ebike_telemetry");
  Serial.println("Alerts   : on state change -> ebike_alerts");
#else
  Serial.println("Supabase : disabled");
#endif
#if USE_TELEGRAM
  bool tgReady = (String(TELEGRAM_TOKEN) != "PASTE_BOT_TOKEN_HERE");
  Serial.print("Telegram : ");
  Serial.println(tgReady ? "configured" : "TOKEN NOT SET - messages disabled");
#else
  Serial.println("Telegram : disabled");
#endif
  Serial.println("The dashboard reads from Supabase, so the browser and");
  Serial.println("this device do not need to be on the same network.");
  Serial.println("-------------------------------------");
  Serial.println();
}

// ============================================================
//  MAIN LOOP
// ============================================================
void loop() {
  serviceBuzzer();
  serviceGPS();

  if (millis() - lastRead >= READ_INTERVAL) {
    lastRead = millis();
    readSensors();
    updateParkingState();
    evaluateStatus();
    updateOutputs();
    if (millis() % 1000 < READ_INTERVAL) publishSerial();
  }

  // Dispatch a queued alert once the buzzer has finished, so the audible
  // warning is never cut short by a blocking HTTPS request.
  if (pending.active && buzzerIdle()) {
    Status  lvl = pending.level;
    String  cd  = pending.code;
    String  rsn = pending.reason;
    pending.active = false;

    if (lvl == NORMAL) {
#if USE_TELEGRAM
      telegramSend(String("\xE2\x9C\x85 *Cleared* - `") + DEVICE_ID +
                   "` is back to normal.");
#endif
    } else {
#if USE_SUPABASE
      uploadAlert(lvl, cd, rsn);
#endif
#if USE_TELEGRAM
      telegramAlert(lvl, cd, rsn);
#endif
    }
  }

  // A sensor that dropped out is retried, so pushing a loose jumper back
  // in brings it back without a reboot.
  if ((!mpuOk || !bmpOk || !inaOk) && millis() - lastProbe >= REPROBE_MS) {
    lastProbe = millis();
    if (!mpuOk && mpuInit()) {
      mpuOk = true;
      Serial.println("[SENSOR] MPU6050 reconnected - recalibrating, keep still");
      calibrateOrientation();
      stillSince = millis();
      if (mpuOk) queueHw("MPU6050 reconnected. Motion alerts are back on.");
    }
    if (!bmpOk && bmpInit()) {
      bmpOk = true;
      Serial.println("[SENSOR] BMP280 reconnected");
      queueHw("BMP280 reconnected. Ambient readings are back.");
```

```
    }
    if (!inaOk && inaInit()) {
      inaOk = true;
      Serial.println("[SENSOR] INA219 reconnected");
      queueHw("INA219 reconnected. Voltage and current are back.");
    }
  }

  if (pendingHw.length() && !pending.active && buzzerIdle()) {
    String msg = pendingHw;
    pendingHw = "";
#if USE_TELEGRAM
    telegramSend(String("\xF0\x9F\x94\xA7 *Hardware* - `") + DEVICE_ID + "`\n" + msg);
#endif
  }

#if USE_SUPABASE
  if (millis() - lastUpload >= SUPABASE_INTERVAL) {
    lastUpload = millis();
    uploadTelemetry();
  }
#endif
}

// ============================================================
//  SENSORS
// ============================================================
// A DS18B20 that does not answer returns -127. Uploading that value
// pollutes the dashboard charts and could mask a real reading, so a bad
// sample is rejected and the last good value is held instead.
float lastGoodMotor = NAN, lastGoodBatt = NAN;
uint16_t sensorFaults = 0;

float validateTemp(float v, float &lastGood, const char* which) {
  if (v > -50.0 && v < 150.0) { lastGood = v; return v; }
  sensorFaults++;
  Serial.print("[SENSOR] ");
  Serial.print(which);
  Serial.print(" read failed (");
  Serial.print(v, 1);
  Serial.println(") - check wiring and the 4.7k pull-up");
  return lastGood;          // NAN until a first good reading arrives
}

void readSensors() {
  sensors.requestTemperatures();
  r.motorTemp   = validateTemp(sensors.getTempC(addrMotor),   lastGoodMotor, "motor
sensor");
  r.batteryTemp = validateTemp(sensors.getTempC(addrBattery), lastGoodBatt,  "battery
sensor");

  float ax = 0, ay = 0, az = 0;
  bool motionValid = false;
  if (mpuOk) {
    if (mpuRead(ax, ay, az)) {
      mpuFailStreak = 0;
      motionValid = true;
    } else if (++mpuFailStreak >= 3) {
      mpuOk = false;
      parkingArmed = false;
      Serial.println("[SENSOR] MPU6050 stopped responding - motion alerts OFF");
      queueHw("MPU6050 stopped responding. Vibration, impact and parking alerts are off
until it reconnects. Check its VCC, GND, SDA and SCL wires.");
    }
  }

  if (!motionValid) {
    r.vibration = NAN;
    r.tiltAngle = NAN;
    r.impactDetected = false;
  } else {

  // Vibration as the magnitude of the DIFFERENCE from the resting vector.
  // Catches sideways and rotational movement that leaves the total
  // magnitude unchanged, which a scalar comparison would miss.
  float dx = ax - refX, dy = ay - refY, dz = az - refZ;
  r.vibration = sqrt(dx * dx + dy * dy + dz * dz);

  // Tilt as the angle between the current gravity vector and the boot
```

```
  // reference - independent of how the board is mounted.
  float magNow = sqrt(ax * ax + ay * ay + az * az);
  float magRef = sqrt(refX * refX + refY * refY + refZ * refZ);
  if (magNow > 0.1 && magRef > 0.1) {
    float dot = (ax * refX + ay * refY + az * refZ) / (magNow * magRef);
    dot = constrain(dot, -1.0, 1.0);
    r.tiltAngle = acos(dot) * 180.0 / PI;
  } else {
    r.tiltAngle = 0;
  }

  r.impactDetected = (r.vibration > IMPACT_THRESHOLD);
  if (r.impactDetected) impactUntil = millis() + IMPACT_LATCH_MS;
  }

  r.ambientTemp = NAN;
  r.pressure    = NAN;
  if (bmpOk) {
    float t = bmp.readTemperature();
    float pr = bmp.readPressure() / 100.0F;
    bool sane = !isnan(t) && t > -40 && t < 85 && !isnan(pr) && pr > 300 && pr < 1100;
    if (sane) {
      bmpFailStreak = 0;
      r.ambientTemp = t;
      r.pressure = pr;
    } else if (++bmpFailStreak >= 3) {
      bmpOk = false;
      Serial.println("[SENSOR] BMP280 stopped responding");
      queueHw("BMP280 stopped responding. Ambient temperature and pressure are unavailable
until it reconnects.");
    }
  }

  // ---- INA219 ----
  r.voltage = NAN; r.current = NAN; r.power = NAN; r.noSupply = false;
  if (inaOk) {
    if (!i2cPresent(inaAddr)) {
      if (++inaFailStreak >= 3) {
        inaOk = false;
        Serial.println("[SENSOR] INA219 stopped responding");
        queueHw("INA219 stopped responding. Voltage, current and power are off until it
reconnects.");
      }
    } else {
      inaFailStreak = 0;
      // Load voltage = bus voltage + the drop across the shunt resistor.
      float v  = ina->getBusVoltage_V() + ina->getShuntVoltage_mV() / 1000.0;
      float mA = ina->getCurrent_mA();
      if (v >= 0 && v < 33.0) {
        r.voltage  = v;
        r.current  = mA;
        r.power    = v * mA / 1000.0;           // watts
        r.noSupply = v < VOLT_NO_SUPPLY;
        learnSupply(v);
      }
    }
  }

  // ---- NEO-6M GPS ----
  r.gpsFix = gpsOk && gps.location.isValid() && gps.location.age() < GPS_FIX_MAX_AGE;
  r.sats   = gps.satellites.isValid() ? (int)gps.satellites.value() : 0;
  r.hdop   = gps.hdop.isValid() ? (float)gps.hdop.hdop() : NAN;
  r.lat = r.lng = NAN;
  r.speed = r.altitude = r.geoDistance = NAN;
  if (r.gpsFix) {
    r.lat = gps.location.lat();
    r.lng = gps.location.lng();
    if (gps.speed.isValid())    r.speed    = gps.speed.kmph();
    if (gps.altitude.isValid()) r.altitude = gps.altitude.meters();
    lastFixLat = r.lat; lastFixLng = r.lng; lastFixAt = millis();

    // Distance ridden this session, counted only while actually moving
    // so GPS jitter at a standstill does not add up.
    bool goodFix = !isnan(r.hdop) && r.hdop <= GPS_MAX_HDOP;
    if (goodFix && !isnan(r.speed) && r.speed > 3.0) {
      if (!isnan(tripLat)) {
        double d = TinyGPSPlus::distanceBetween(tripLat, tripLng, r.lat, r.lng);
        if (d > 5.0 && d < 500.0) tripKm += d / 1000.0;
      }
```

```
      tripLat = r.lat; tripLng = r.lng;
    }
  }

  r.timestamp = millis();
}

void updateParkingState() {
  if (isnan(r.vibration)) {         // no IMU: parking protection cannot work
    parkingArmed = false;
    stillSince = millis();
    return;
  }
  if (r.vibration < IDLE_VIBRATION) {
    if (!parkingArmed && millis() - stillSince > ARM_DELAY) {
      parkingArmed = true;
      Serial.println("[PARKING] armed - vehicle has been still");
    }
  } else {
    stillSince = millis();
  }
}

// ============================================================
//  ALERT LOGIC
// ============================================================

// Confirm over several reads, clear at a lower value than it triggers.
bool confirm(bool over, bool under, uint8_t &counter, bool &flag) {
  if (!flag) {
    if (over) { if (++counter >= CONFIRM_SAMPLES) { flag = true; counter = 0; } }
    else counter = 0;
  } else {
    if (under) { if (++counter >= CONFIRM_SAMPLES) { flag = false; counter = 0; } }
    else counter = 0;
  }
  return flag;
}

void evaluateStatus() {
  // isnan guards keep a missing sensor from being read as 0 degrees.
  bool tempOk = !isnan(r.motorTemp), battOk = !isnan(r.batteryTemp);
  if (!tempOk) { fMotorW = fMotorC = false; }
  if (!battOk) { fBattW  = fBattC  = false; }

  if (tempOk) {
  confirm(r.motorTemp   > TEMP_MOTOR_WARN, r.motorTemp   < TEMP_MOTOR_WARN_CLR, cMotorW,
fMotorW);
  confirm(r.motorTemp   > TEMP_MOTOR_CRIT, r.motorTemp   < TEMP_MOTOR_CRIT_CLR, cMotorC,
fMotorC);
  }
  if (battOk) {
  confirm(r.batteryTemp > TEMP_BATT_WARN,  r.batteryTemp < TEMP_BATT_WARN_CLR,  cBattW,
fBattW);
  confirm(r.batteryTemp > TEMP_BATT_CRIT,  r.batteryTemp < TEMP_BATT_CRIT_CLR,  cBattC,
fBattC);
  }
  bool motionOk = !isnan(r.vibration);
  if (motionOk) {
    confirm(r.vibration > VIBRATION_WARN, r.vibration < VIBRATION_WARN_CLR, cVibW, fVibW);
    confirm(r.vibration > VIBRATION_CRIT, r.vibration < VIBRATION_CRIT_CLR, cVibC, fVibC);
  } else {
    fVibW = fVibC = false;
    cVibW = cVibC = 0;
    impactUntil = 0;
  }

  bool parkMove = motionOk && parkingArmed &&
                  (r.vibration > VIBRATION_WARN || r.tiltAngle > TILT_THRESHOLD);
  if (parkMove) { if (cParkMove < 255) cParkMove++; } else cParkMove = 0;
  // ---- Power ----
  bool elecOk = !isnan(r.voltage) && !r.noSupply && voltNominal > 0;
  if (elecOk) {
    confirm(r.voltage < voltNominal * VOLT_WARN_PCT, r.voltage > voltNominal *
VOLT_WARN_CLR_PCT, cVoltW, fVoltW);
    confirm(r.voltage < voltNominal * VOLT_CRIT_PCT, r.voltage > voltNominal *
VOLT_CRIT_CLR_PCT, cVoltC, fVoltC);
  } else { fVoltW = fVoltC = false; cVoltW = cVoltC = 0; }
  if (!isnan(r.current)) {
```

```
    confirm(r.current > CURRENT_MAX_WARN, r.current < CURRENT_MAX_WARN * 0.9, cCurW, fCurW);
    confirm(r.current > CURRENT_MAX_CRIT, r.current < CURRENT_MAX_CRIT * 0.9, cCurC, fCurC);
  } else { fCurW = fCurC = false; cCurW = cCurC = 0; }

  // ---- GPS: overspeed and geofence, only on a trustworthy fix ----
  bool goodFix = r.gpsFix && !isnan(r.hdop) && r.hdop <= GPS_MAX_HDOP;
  if (goodFix && !isnan(r.speed)) {
    confirm(r.speed > OVERSPEED_KMH, r.speed < OVERSPEED_CLR_KMH, cOver, fOver);
  } else { fOver = false; cOver = 0; }

  bool geoTheft = false;
  if (!parkingArmed) {
    parkLat = parkLng = NAN;
    cGeo = 0;
  } else if (goodFix) {
    if (isnan(parkLat)) { parkLat = r.lat; parkLng = r.lng; }   // parked here
    r.geoDistance = TinyGPSPlus::distanceBetween(parkLat, parkLng, r.lat, r.lng);
    if (r.geoDistance > GEOFENCE_M) { if (cGeo < 255) cGeo++; } else cGeo = 0;
    geoTheft = cGeo >= CONFIRM_SAMPLES;
  }

  bool theft  = (cParkMove >= CONFIRM_SAMPLES) || geoTheft;
  bool impact = (millis() < impactUntil);

  Status s = NORMAL;
  String why = "All systems normal", code = "OK";

  if (fMotorW) { s = WARNING; why = "Motor temperature elevated";   code =
"MOTOR_TEMP_WARN"; }
  if (fBattW)  { s = WARNING; why = "Battery temperature elevated"; code = "BATT_TEMP_WARN";
 }
  if (fVibW)   { s = WARNING; why = "Abnormal vibration";           code = "VIB_WARN"; }
  if (fVoltW) { s = WARNING; why = "Battery voltage low";  code = "VOLT_WARN"; }
  if (fCurW)  { s = WARNING; why = "Current draw high";    code = "CURRENT_WARN"; }
  if (fOver)  { s = WARNING; why = "Overspeed " + String(r.speed, 0) + " km/h"; code =
"OVERSPEED"; }

  if (fMotorC) { s = CRITICAL; why = "Motor overheating";   code = "MOTOR_TEMP_CRIT"; }
  if (fBattC)  { s = CRITICAL; why = "Battery overheating"; code = "BATT_TEMP_CRIT"; }
  if (fVibC)   { s = CRITICAL; why = "Severe vibration";    code = "VIB_CRIT"; }
  if (fVoltC) { s = CRITICAL; why = "Battery critically low"; code = "VOLT_CRIT"; }
  if (fCurC)  { s = CRITICAL; why = "Current draw critical";  code = "CURRENT_CRIT"; }
  if (theft)   {
    s = CRITICAL; code = "THEFT";
    why = geoTheft ? "Moved " + String(r.geoDistance, 0) + " m while parked"
                   : "Movement while parked";
  }
  if (impact)  { s = CRITICAL; why = "Impact detected";       code = "IMPACT"; }

  r.status = s; r.reason = why; r.code = code;

  // Act once on entry, not on every cycle.
  if (s != prevStatus) {
    if (s == CRITICAL) {
      critCount++;
      // Distinct patterns, so the alert is identifiable by ear alone.
      if      (code == "IMPACT") startBuzzer(2600, 90, 60, 8);
      else if (code == "THEFT")  startBuzzer(2200, 250, 120, 6);
      else                       startBuzzer(2000, 400, 200, 5);
      alertScreenUntil = millis() + 4000;
      Serial.print("[CRITICAL] "); Serial.print(code);
      Serial.print(" - ");         Serial.println(why);
      pending = {true, s, code, why};
    } else if (s == WARNING) {
      warnCount++;
      startBuzzer(1200, 120, 400, 2);
      alertScreenUntil = millis() + 2500;
      Serial.print("[WARNING]  "); Serial.print(code);
      Serial.print(" - ");         Serial.println(why);
      pending = {true, s, code, why};
    } else {
      stopBuzzer();
      Serial.println("[CLEARED]  back to normal");
      pending = {true, NORMAL, String("CLEARED"), String("Back to normal")};
    }
    prevStatus = s;
  }
}
// ============================================================
```

```
//  OUTPUTS
// ============================================================
void updateOutputs() {
  setLEDs(r.status);
  if (millis() < alertScreenUntil) drawAlertScreen();
  else                             drawLiveScreen();
}

void setLEDs(Status s) {
  static bool blink = false;
  blink = !blink;
  if (s == CRITICAL) {
    digitalWrite(PIN_LED_RED, blink);      // flashing red
    digitalWrite(PIN_LED_GREEN, LOW);
  } else if (s == WARNING) {
    digitalWrite(PIN_LED_RED, HIGH);       // both = amber
    digitalWrite(PIN_LED_GREEN, HIGH);
  } else {
    digitalWrite(PIN_LED_RED, LOW);
    digitalWrite(PIN_LED_GREEN, HIGH);
  }
}

const char* statusName(Status s) {
  if (s == CRITICAL) return "CRITICAL";
  if (s == WARNING)  return "WARNING";
  return "NORMAL";
}

// ---- Full-screen alert takeover ----
void drawAlertScreen() {
  static bool phase = false;
  phase = !phase;
  bool crit = (r.status == CRITICAL);

  display.clearDisplay();
  if (crit && phase) {
    display.fillRect(0, 0, 128, 64, SSD1306_WHITE);
    display.setTextColor(SSD1306_BLACK);
  } else {
    display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
    display.drawRect(1, 1, 126, 62, SSD1306_WHITE);
    display.setTextColor(SSD1306_WHITE);
  }
  int fg = (crit && phase) ? SSD1306_BLACK : SSD1306_WHITE;

  display.drawTriangle(20, 30, 8, 48, 32, 48, fg);
  display.drawTriangle(20, 33, 12, 46, 28, 46, fg);
  display.drawLine(20, 37, 20, 42, fg);
  display.drawPixel(20, 44, fg);

  display.setTextSize(2);
  display.setCursor(42, 8);
  display.print(crit ? "ALERT" : "WARN");

  display.setTextSize(1);
  String msg = r.reason;
  int split = msg.indexOf(' ', 11);
  if (split < 0 || msg.length() <= 14) {
    display.setCursor(42, 32);
    display.print(msg);
  } else {
    display.setCursor(42, 30);
    display.print(msg.substring(0, split));
    display.setCursor(42, 40);
    display.print(msg.substring(split + 1));
  }
  display.setCursor(42, 52);
  display.print(r.code);

  display.display();
  display.setTextColor(SSD1306_WHITE);
}

// ---- Animated live screen ----
uint8_t vibHistory[64];
uint8_t vibIndex = 0;
float wheelAngle = 0;
bool blinkPhase = false;
```

```
void drawLiveScreen() {
  blinkPhase = !blinkPhase;
  wheelAngle += 0.35 + r.vibration * 0.25;

  float scaled = isnan(r.vibration) ? 0 : r.vibration * (14.0 / VIBRATION_CRIT);
  if (scaled > 14) scaled = 14;
  vibHistory[vibIndex] = (uint8_t)scaled;
  vibIndex = (vibIndex + 1) % 64;

  display.clearDisplay();

  bool flash = (r.status == CRITICAL && blinkPhase);
  if (flash) display.fillRect(0, 0, 128, 13, SSD1306_WHITE);
  else       display.drawRect(0, 0, 128, 13, SSD1306_WHITE);

  int fg = flash ? SSD1306_BLACK : SSD1306_WHITE;
  display.setTextColor(fg);
  display.setTextSize(1);
  display.setCursor(20, 3);
  display.print(statusName(r.status));
  if (parkingArmed) display.print(" PARK");

  if (WiFi.status() == WL_CONNECTED) {
    for (int b = 0; b < 3; b++)
      display.fillRect(112 + b * 4, 8 - b * 3, 3, 3 + b * 3, fg);
#if USE_SUPABASE
    // Filled dot = last cloud upload succeeded, hollow = some failed.
    if (uploadOk > 0 && uploadFail == 0) display.fillCircle(105, 6, 2, fg);
    else if (uploadOk > 0)               display.drawCircle(105, 6, 2, fg);
#endif
  } else {
    display.setCursor(112, 3);
    display.print("--");
  }
  drawSmallWheel(9, 6, 4, wheelAngle, fg);
  display.setTextColor(SSD1306_WHITE);

  display.setCursor(0, 17);
  display.print("MOT ");    display.print(r.motorTemp, 1);
  display.print("C  BAT "); display.print(r.batteryTemp, 1); display.println("C");
  // The middle line rotates every 2 s: ambient, power, GPS.
  display.setCursor(0, 27);
  switch ((millis() / 2000) % 3) {
    case 0:
      if (isnan(r.ambientTemp)) display.print("BMP280 MISSING");
      else {
        display.print("AMB "); display.print(r.ambientTemp, 1);
        display.print("C "); display.print(r.pressure, 0); display.print("hPa");
      }
      break;
    case 1:
      if (!inaOk)          display.print("INA219 MISSING");
      else if (r.noSupply) display.print("PWR NO SUPPLY");
      else {
        display.print(r.voltage, 2); display.print("V ");
        display.print(r.current / 1000.0, 2); display.print("A ");
        display.print(r.power, 1); display.print("W");
      }
      break;
    default:
      if (!gpsOk)          display.print("GPS MISSING");
      else if (!r.gpsFix) { display.print("GPS NO FIX "); display.print(r.sats);
display.print(" SAT"); }
      else {
        display.print("GPS "); display.print(r.sats); display.print("sat ");
        display.print(isnan(r.speed) ? 0.0 : r.speed, 0); display.print("km/h");
      }
  }
  display.setCursor(0, 37);
  if (isnan(r.vibration)) {
    display.print("MPU6050 MISSING");
  } else {
    display.print("VIB ");    display.print(r.vibration, 2);
    display.print("  TLT ");  display.print(r.tiltAngle, 0); display.println((char)247);
  }

  display.drawLine(0, 63, 127, 63, SSD1306_WHITE);
  for (int i = 0; i < 64; i++) {
    uint8_t h = vibHistory[(vibIndex + i) % 64];
```

```
    if (h > 0) display.drawLine(i * 2, 63 - h, i * 2, 62, SSD1306_WHITE);
  }
  int warnY = 63 - (int)(VIBRATION_WARN * (14.0 / VIBRATION_CRIT));
  for (int x = 0; x < 128; x += 6) display.drawPixel(x, warnY, SSD1306_WHITE);

  display.display();
}

void drawSmallWheel(int cx, int cy, int radius, float angle, int color) {
  display.drawCircle(cx, cy, radius, color);
  for (int s = 0; s < 4; s++) {
    float a = angle + s * PI / 2.0;
    display.drawLine(cx, cy, cx + cos(a) * radius, cy + sin(a) * radius, color);
  }
}

// ---- Boot animation ----
void bootAnimation() {
  for (int x = -20; x < 148; x += 6) {
    display.clearDisplay();
    drawWheel(x, 38, 12, x * 0.15);
    display.drawLine(0, 52, 127, 52, SSD1306_WHITE);
    display.display();
    delay(16);
  }
  for (int x = -90; x <= 14; x += 7) {
    display.clearDisplay();
    display.setTextSize(2);
    display.setCursor(x, 16);
    display.print("E-BIKE");
    display.display();
    delay(14);
  }
  const char* sub = "SMART MONITOR";
  for (int i = 0; i <= (int)strlen(sub); i++) {
    display.clearDisplay();
    display.setTextSize(2);
    display.setCursor(14, 16);
    display.print("E-BIKE");
    display.setTextSize(1);
    display.setCursor(16, 40);
    for (int j = 0; j < i; j++) display.print(sub[j]);
    display.display();
    delay(45);
  }
  delay(350);
  for (int i = 0; i < 5; i++) {
    display.drawRect(i * 2, i * 2, 128 - i * 4, 64 - i * 4, SSD1306_WHITE);
    display.display();
    delay(35);
  }
  delay(200);
}

void drawWheel(int cx, int cy, int radius, float angle) {
  display.drawCircle(cx, cy, radius, SSD1306_WHITE);
  display.drawCircle(cx, cy, 2, SSD1306_WHITE);
  for (int s = 0; s < 6; s++) {
    float a = angle + s * PI / 3.0;
    display.drawLine(cx, cy, cx + cos(a) * radius, cy + sin(a) * radius, SSD1306_WHITE);
  }
}

void calibrationScreen() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(4, 6);
  display.println("LEARNING MOUNTING");
  display.setCursor(4, 20);
  display.println("Keep the board still");
  display.setCursor(4, 30);
  display.println("in its fitted position");
  display.drawRect(10, 44, 108, 10, SSD1306_WHITE);
  display.display();
  for (int i = 0; i <= 104; i += 4) {
    display.fillRect(12, 46, i, 6, SSD1306_WHITE);
    display.display();
    delay(4);
  }
```

```
}

void readySplash() {
  display.clearDisplay();
  display.setTextSize(2);
  display.setCursor(22, 8);
  display.print("READY");
  display.setTextSize(1);
  display.setCursor(4, 32);
  display.print("WiFi: ");
  display.println(WiFi.status() == WL_CONNECTED ? "connected" : "offline");
  if (WiFi.status() == WL_CONNECTED) {
    display.setCursor(4, 44);
    display.println(WiFi.localIP());
    display.setCursor(4, 54);
    display.print("Cloud: ");
    display.println(DEVICE_ID);
  } else {
    display.setCursor(4, 44);
    display.println("No cloud upload");
  }
  display.display();
  delay(2500);
}

// ============================================================
//  SUPABASE
// ============================================================
int supabasePost(const char* table, const String& body) {
  lastHttpBody = "";
  if (WiFi.status() != WL_CONNECTED) return -1;

  WiFiClientSecure client;
  // No certificate pinning. Acceptable for a prototype on a trusted
  // network; pin the Supabase root certificate beyond the demo.
  client.setInsecure();
  client.setTimeout(8);

  HTTPClient http;
  String url = String(SUPABASE_URL) + "/rest/v1/" + table;
  if (!http.begin(client, url)) return -2;

  http.addHeader("Content-Type", "application/json");
  http.addHeader("apikey", SUPABASE_KEY);
  http.addHeader("Authorization", String("Bearer ") + SUPABASE_KEY);
  http.addHeader("Prefer", "return=minimal");
  http.setTimeout(8000);

  int code = http.POST(body);
  if (code <= 0) {
    Serial.print("[SUPABASE] transport error: ");
    Serial.println(http.errorToString(code));
  } else if (code >= 300) {
    lastHttpBody = http.getString();
    Serial.print("[SUPABASE] HTTP "); Serial.print(code);
    Serial.print(" - ");              Serial.println(lastHttpBody);
  }
  http.end();
  return code;
}

// Supabase answers PGRST204 when the payload names a column the table does
// not have, and then rejects the WHOLE row.
bool missingColumn(int code) {
  return code == 400 && (lastHttpBody.indexOf("PGRST204") >= 0 ||
                         lastHttpBody.indexOf("Could not find") >= 0);
}

void uploadTelemetry() {
  StaticJsonDocument<1024> doc;
  doc["device_id"]     = DEVICE_ID;
  if (!isnan(r.motorTemp))   doc["motor_temp"]   = r.motorTemp;
  if (!isnan(r.batteryTemp)) doc["battery_temp"] = r.batteryTemp;
  doc["sensor_faults"] = sensorFaults;
  doc["mpu_ok"]        = mpuOk;
  doc["bmp_ok"]        = bmpOk;
  if (!isnan(r.ambientTemp)) doc["ambient_temp"] = r.ambientTemp;
  if (!isnan(r.pressure))    doc["pressure"]     = r.pressure;
  if (!isnan(r.vibration)) {
```

```
    doc["vibration_ms2"] = r.vibration;
    doc["vibration_g"]   = r.vibration / 9.81;
  }
  if (!isnan(r.tiltAngle))   doc["tilt"]         = r.tiltAngle;
  doc["impact"]        = (millis() < impactUntil);
  doc["parking_armed"] = parkingArmed;
  doc["status"]        = statusName(r.status);
  doc["code"]          = r.code;
  doc["reason"]        = r.reason;
  doc["uptime_ms"]     = millis();
  if (!isnan(r.voltage)) doc["voltage"]   = r.voltage;
  if (!isnan(r.current)) doc["current_a"] = r.current / 1000.0;

  // Core fields above exist in every version of the table. The block
  // below needs the GPS/power columns (add_gps_power.sql).
  String core;
  serializeJson(doc, core);

  if (extendedSchema) {
    doc["ina_ok"]  = inaOk;
    doc["gps_ok"]  = gpsOk;
    doc["gps_fix"] = r.gpsFix;
    if (!isnan(r.power)) doc["power_w"] = r.power;
    if (gpsOk) {
      doc["satellites"] = r.sats;
      if (!isnan(r.hdop)) doc["hdop"] = r.hdop;
    }
    if (r.gpsFix) {
      doc["lat"] = r.lat;
      doc["lng"] = r.lng;
      if (!isnan(r.speed))    doc["speed_kmh"]  = r.speed;
      if (!isnan(r.altitude)) doc["altitude_m"] = r.altitude;
    }
    String body;
    serializeJson(doc, body);
    int code = supabasePost("ebike_telemetry", body);
    if (code >= 200 && code < 300) { uploadOk++; return; }
    if (!missingColumn(code)) { uploadFail++; return; }
    extendedSchema = false;
    Serial.println("[SUPABASE] table has no GPS/power columns - run add_gps_power.sql.");
    Serial.println("[SUPABASE] sending the original fields only until then.");
    queueHw("Supabase is missing the GPS/power columns, so location and power are not being
stored. Run add_gps_power.sql in the Supabase SQL editor, then reset the ESP32.");
  }

  int code = supabasePost("ebike_telemetry", core);
  if (code >= 200 && code < 300) uploadOk++;
  else                           uploadFail++;
}

void uploadAlert(Status level, const String& code, const String& reason) {
  StaticJsonDocument<512> doc;
  doc["device_id"]     = DEVICE_ID;
  doc["level"]         = statusName(level);
  doc["code"]          = code;
  doc["reason"]        = reason;
  doc["motor_temp"]    = r.motorTemp;
  doc["battery_temp"]  = r.batteryTemp;
  if (!isnan(r.vibration)) doc["vibration_ms2"] = r.vibration;
  if (!isnan(r.tiltAngle)) doc["tilt"]          = r.tiltAngle;
  doc["notified_telegram"] = USE_TELEGRAM;

  String body;
  serializeJson(doc, body);
  int http = -1;
  if (extendedSchema && r.gpsFix) {
    doc["lat"] = r.lat;
    doc["lng"] = r.lng;
    String withPos;
    serializeJson(doc, withPos);
    http = supabasePost("ebike_alerts", withPos);
    if (missingColumn(http)) http = supabasePost("ebike_alerts", body);
  } else {
    http = supabasePost("ebike_alerts", body);
  }
  Serial.print("[SUPABASE] alert ");
  Serial.print(code);
  Serial.println((http >= 200 && http < 300) ? " uploaded" : " FAILED");
}
```

```
// ============================================================
//  TELEGRAM
// ============================================================
bool telegramSend(const String& text) {
  if (WiFi.status() != WL_CONNECTED) return false;
  if (String(TELEGRAM_TOKEN) == "PASTE_BOT_TOKEN_HERE") return false;

  WiFiClientSecure client;
  client.setInsecure();
  client.setTimeout(8);

  HTTPClient http;
  String url = String("https://api.telegram.org/bot") + TELEGRAM_TOKEN + "/sendMessage";
  if (!http.begin(client, url)) { tgFail++; return false; }

  StaticJsonDocument<768> doc;
  doc["chat_id"]    = TELEGRAM_CHAT_ID;
  doc["text"]       = text;
  doc["parse_mode"] = "Markdown";
  String body;
  serializeJson(doc, body);

  http.addHeader("Content-Type", "application/json");
  http.setTimeout(8000);
  int code = http.POST(body);

  bool ok = (code >= 200 && code < 300);
  if (ok) tgOk++;
  else {
    tgFail++;
    Serial.print("[TELEGRAM] failed, HTTP ");
    Serial.print(code);
    if (code > 0) { Serial.print(" - "); Serial.print(http.getString()); }
    Serial.println();
  }
  http.end();
  return ok;
}

void telegramAlert(Status level, const String& code, const String& reason) {
  // Critical alerts always send. Repeats of the SAME warning are held back
  // for a minute, so a reading hovering near a limit cannot spam the chat.
  bool isCritical = (level == CRITICAL);
  if (!isCritical && code == lastTgCode && millis() - lastTgTime < TELEGRAM_COOLDOWN) {
    Serial.println("[TELEGRAM] suppressed (cooldown)");
    return;
  }
  lastTgCode = code;
  lastTgTime = millis();

  String icon = isCritical ? "\xF0\x9F\x9A\xA8" : "\xE2\x9A\xA0";   // siren / warning
  String msg  = icon + " *" + statusName(level) + "* - " + reason + "\n\n";
  msg += "Device: `" + String(DEVICE_ID) + "`\n";
  msg += "Code: `" + code + "`\n";
  msg += "Motor temp: *" + String(r.motorTemp, 1) + " C*\n";
  msg += "Battery temp: *" + String(r.batteryTemp, 1) + " C*\n";
  if (!isnan(r.ambientTemp)) msg += "Ambient: " + String(r.ambientTemp, 1) + " C\n";
  if (!isnan(r.vibration))   msg += "Vibration: *" + String(r.vibration, 2) + " m/s2*\n";
  if (!isnan(r.tiltAngle))   msg += "Tilt: " + String(r.tiltAngle, 0) + " deg\n";
  if (!isnan(r.voltage) && !r.noSupply) {
    msg += "Power: " + String(r.voltage, 2) + " V, " + String(r.current, 0) + " mA, " +
           String(r.power, 2) + " W\n";
  }
  if (r.gpsFix && !isnan(r.speed)) msg += "Speed: " + String(r.speed, 0) + " km/h\n";
  if (parkingArmed) msg += "Parking mode: *armed*\n";
  // Where it happened - the most useful line in a theft or crash alert.
  if (r.gpsFix) {
    msg += "\nLocation: " + mapsLink(r.lat, r.lng) + "\n";
  } else if (!isnan(lastFixLat)) {
    msg += "\nLast known location (" + String((millis() - lastFixAt) / 60000) +
           " min ago): " + mapsLink(lastFixLat, lastFixLng) + "\n";
  } else if (gpsOk) {
    msg += "\nLocation: no GPS fix yet\n";
  }

  if (code == "IMPACT")           msg += "\n_Possible collision or fall. Inspect the
vehicle._";
  else if (code == "THEFT")       msg += "\n_The vehicle moved while parked. Check
immediately._";
```

```
  else if (code == "OVERSPEED")    msg += "\n_Above the 25 km/h limit for low-speed
e-bikes._";
  else if (code == "VOLT_CRIT")    msg += "\n_Battery nearly flat - stop and recharge._";
  else if (code == "CURRENT_CRIT") msg += "\n_Abnormal current draw - check for a short or
stalled motor._";
  else if (code.endsWith("CRIT"))  msg += "\n_Stop use and allow the vehicle to cool._";
  else                             msg += "\n_Monitor this reading; inspect if it
persists._";

  telegramSend(msg);
}

// ============================================================
//  SERIAL DEBUG OUTPUT
// ============================================================
String buildJSON() {
  StaticJsonDocument<1024> doc;
  doc["motor_temp"]    = r.motorTemp;
  doc["battery_temp"]  = r.batteryTemp;
  doc["mpu_ok"]        = mpuOk;
  doc["bmp_ok"]        = bmpOk;
  if (!isnan(r.ambientTemp)) doc["ambient_temp"] = r.ambientTemp;
  if (!isnan(r.pressure))    doc["pressure"]     = r.pressure;
  if (!isnan(r.vibration)) {
    doc["vibration_ms2"] = r.vibration;
    doc["vibration_g"]   = r.vibration / 9.81;
  }
  if (!isnan(r.tiltAngle))   doc["tilt"]         = r.tiltAngle;
  doc["impact"]        = (millis() < impactUntil);
  doc["parking_armed"] = parkingArmed;
  doc["status"]        = statusName(r.status);
  doc["code"]          = r.code;
  doc["reason"]        = r.reason;
  doc["warn_events"]   = warnCount;
  doc["crit_events"]   = critCount;
  doc["cloud_ok"]      = uploadOk;
  doc["cloud_fail"]    = uploadFail;
  doc["telegram_ok"]   = tgOk;
  doc["telegram_fail"] = tgFail;
  doc["uptime_ms"]     = millis();
  if (!isnan(r.voltage)) doc["voltage"]   = r.voltage;
  if (!isnan(r.current)) doc["current_a"] = r.current / 1000.0;
  if (!isnan(r.power))   doc["power_w"]   = r.power;
  doc["ina_ok"]  = inaOk;
  doc["gps_ok"]  = gpsOk;
  doc["gps_fix"] = r.gpsFix;
  doc["sats"]    = r.sats;
  if (r.gpsFix) {
    doc["lat"] = r.lat;
    doc["lng"] = r.lng;
    if (!isnan(r.speed)) doc["speed_kmh"] = r.speed;
  }
  doc["trip_km"] = tripKm;

  String out;
  serializeJson(doc, out);
  return out;
}

void publishSerial() {
  Serial.println(buildJSON());
}
```